

Probabilistic Analysis And Randomized Algo:

Probabilistic Analysis: Determine the running time of an algorithm by assuming an inherent probability distribution for the input. Eg: assuming all input permutation are equally likely.

Randomized Algorithm: The algorithm's behavior is determined not just by input, but by values produced by a random-number generator during execution.

But Why We are Interested
in
Randomized Algorithm?

Motivation: Randomization is a powerful tool to enforce a distribution on inputs, ensuring that no specific **bad** input always causes poor performance.

Good Thing Comes With Cost

Cost of Randomization: Random Number generator (RNG) has a cost.

The Hiring Problem

You need to hire a office assistant by interviewing a series of candidate.

→ Committed to have best possible candidate all the time.

→ You interview one by one. If the current candidate is better than the current assistant you immediately fire him and recruit current candidate.

Interview Cost = C_i

Fire and hire Cost = C_h

You will interview n candidates that may cause m hiring.

Total Cost: $(nC_i + mC_h)$

General Task: Find maximum or minimum value in a sequence by examining one by one maintaining a current winner.

Hire Assistant: Worst Case

→ Candidates are coming in increasing order of their quality. Cost $O(nc_n)$.

But does it always happen?
What is the Average Case?

Here Comes the Probabilistic Analysis:

- Make assumption on the input distribution.
- Get average or expected value over the input distribution.

Average Case running time

Hiring Assistant: Random Order Assumption:

→ Total order on Candidate: Any two candidate can be compared and a clear decision can be made on who is better. Candidate a or b :

$$a < b \text{ or } a > b \text{ or } a = b$$

→ Candidate Ranking: Each candidate assigned a unique rank from 1 to n . Higher rank better quality.

→ Uniform Random Permutation:

The ordered list $\text{rank}(1), \text{rank}(2), \text{rank}(3) \dots \text{rank}(n)$ is a permutation of $\langle 1, 2, 3, \dots, n \rangle$.

→ Random Order: Candidate may come in any Random order saying rank list is equally likely over the $n!$ possible permutation.

There Hiring problem Can have a probabilistic Analysis.

From Probabilistic Analysis to Randomized Algorithm

→ The problem with passive randomness:

→ Assumption candidate come in random order.

→ Reality: Impossible to know if they are truly random.

→ Enforce Randomness: Control the order of events.

→ Receive the list of n candidate.

→ Randomly chose who to interview.

Instead of Hoping to have random
distribution Enforce randomness.

Average Case Vs Expected Running Time.

→ Average case RT: The probability distribution is over the inputs to the algorithm.

→ Expected RT: The probability distribution is over the random choices made by the algorithm itself.

Randomized Algorithm provides performance guarantees that do not depend on specific distribution of input.

Indicator Random Variable

Given a sample space S and an event $A \in S$

Indicator Random Variable $I\{A\}$.

$$I\{A\} = \begin{cases} 1 & \text{if } A \text{ occurs} \\ 0 & \text{if } A \text{ doesn't occur.} \end{cases}$$

Example: Expected number of Head for coin flip.

$$S = \{H, T\}, \quad P(H) = P(T) = 1/2$$

Indicator Random Variable X_H for event H

$$\begin{aligned} X_H &= I\{H\} \\ &= \begin{cases} 1 & \text{if } H \text{ occurs} \\ 0 & \text{if } T \text{ occurs} \end{cases} \end{aligned}$$

Expect number of H :

$$\begin{aligned} E[X_H] &= 1 \cdot P(H) + 0 \cdot P(T) \\ &= 1/2 \end{aligned}$$

Lemma 5.1: For a sample space S and event $A \in \mathcal{S}$
let $X_A = I\{A\}$, then $E[X_A] = P(A)$

Proof: By Defⁿ of Indicator random variable

$$\begin{aligned} E[X_A] &= E[I\{A\}] = 1 \cdot P(A) + 0 \cdot P(\bar{A}) \\ &= P(A) \end{aligned}$$

Here $\bar{A} = S - A$

Expected number of Head in n coin flips:

let $X_i = I\{i^{\text{th}} \text{ flip results in } H\}$

let X is the random variable for number of head
in n flips

$$\begin{aligned} X &= \sum_{i=1}^n X_i \\ E[X] &= E\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n E[X_i] \\ &= \sum_{i=1}^n 1/2 = n/2 \end{aligned}$$

Analysis of Hiring Problem with Indicator RV.

⇒ Expected number of hires:

Let X is a rv = number of times you hire.

$$E[X] = \sum_{x=1}^n x P(X=x).$$

But how to Calculate?

⇒ Using Indicator RV:

Let x_i a rv for i^{th} candidate being hired

$$x_i = I\{\text{candidate } i \text{ is hired}\}$$

$$= \begin{cases} 1 & \text{if candidate } i \text{ is hired} \\ 0 & \text{if candidate } i \text{ is not hired} \end{cases}$$

$$\therefore X = x_1 + x_2 + x_3 + \dots + x_n$$

$$E[X] = E\left[\sum_{i=1}^n x_i\right]$$

$$= \sum_{i=1}^n E[x_i]$$

Analysis of Hiring Problem with Indicator RV.

$$E[X_i] = \underbrace{P(X_i)}_{??} \text{ by Lemma 5.1}$$

→ Hiring Criteria: Candidate i is hired only if i is better than all candidate previously interview (1 through $i-1$)

→ Assumption of Randomness: candidate list is a uniform random permutation. So first i candidate are in random order.

→ In random ordering of i distinct element any of them can be best qualified equally likely.

$$\text{Therefore } P(X_i) = 1/i = E[X_i]$$

$$\therefore E[X] = \sum_{i=1}^n 1/i = \ln(n).$$

Average Hiring Cost: $c_h \ln(n)$

Analysis of Randomized Quick Sort:

PARTITION(A, p, r)

```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```

QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2      // Partition the subarray around the pivot, which ends up in  $A[q]$ .
3       $q = \text{PARTITION}(A, p, r)$ 
4      QUICKSORT( $A, p, q - 1$ ) // recursively sort the low side
5      QUICKSORT( $A, q + 1, r$ ) // recursively sort the high side
```

Deterministic Quick Sort.

Worst Case of Deterministic Quick Sort

⇒ Chosen pivot is the maximum or the minimum

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= \Theta(n^2) \end{aligned}$$

Best Case:

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \log n) \end{aligned}$$

Average Case: Any split of constant proportionality still yields $n \log n$. [9:1 or 99:1]

$$\begin{aligned} T(n) &= T(9n/10) + T(n/10) + \Theta(n) \\ &= n \log_{10/9} n \approx n \log n. \end{aligned}$$

Randomized Quick Sort

RANDOMIZED-PARTITION(A, p, r)

- 1 $i = \text{RANDOM}(p, r)$
- 2 exchange $A[r]$ with $A[i]$
- 3 **return** PARTITION(A, p, r)

RANDOMIZED-QUICKSORT(A, p, r)

- 1 **if** $p < r$
- 2 $q = \text{RANDOMIZED-PARTITION}(A, p, r)$
- 3 RANDOMIZED-QUICKSORT($A, p, q - 1$)
- 4 RANDOMIZED-QUICKSORT($A, q + 1, r$)

Difference : Choose pivot Randomly.

Analysis of Randomized Quick Sort:

Lemma 7.1

The running time of QUICKSORT on an n -element array is $O(n + X)$, where X is the number of element comparisons performed.

Proof: Premise $\rightarrow$ Running time is dominated by the Partition Procedure.

$\rightarrow$ Each partition selects a pivot **excludes** it from next partition calls.

$\therefore n$ calls of Partition **at most**.

$\therefore 2n$ calls of QuickSort.

$\rightarrow$ Partition Cost: $O(1)$ plus number iteration in for loop.

$\rightarrow$ Each iteration compares pivot and another element.

$\therefore$ All comparison is proportional to X .

$\therefore O(n + X)$

Use of Indicator Random Variable in Quick Sort

Goal: Compute $E[X]$ to get the expected running time $O(n + X)$.

Lemma 7.2

During the execution of RANDOMIZED-QUICKSORT on an array of n distinct elements $z_1 < z_2 < \dots < z_n$, an element z_i is compared with an element z_j , where $i < j$, if and only if one of them is chosen as a pivot before any other element in the set Z_{ij} . Moreover, no two elements are ever compared twice.

Proof: $Z_{ij} = \{z_i, z_{i+1}, z_{i+2}, \dots, z_j\}$

$\Rightarrow$ First Pivot: $x \in Z_{ij}$

$\Rightarrow$ Two Scenarios: $z_i < x < z_j$ (Scene A)

$\left\{ \begin{array}{l} \therefore z_i \text{ will be on left part of } x \\ \therefore z_j \text{ will be on right part of } x. \end{array} \right.$

z_i and z_j will not be compared.

Scene 13: $x = z_i$ or $x = z_j$

$\therefore$ pivot is compared with all other in Z_{ij}

$\therefore z_i$ is compared with z_j

Since pivot is removed from the array
 z_i and z_j will be compared again.

Lemma 7.3

Consider an execution of the procedure RANDOMIZED-QUICKSORT on an array of n distinct elements $z_1 < z_2 < \dots < z_n$. Given two arbitrary elements z_i and z_j where $i < j$, the probability that they are compared is $2/(j - i + 1)$.

Proof: z_i and z_j are compared only if they are the first pivot in Z_{ij} .

$$\begin{aligned} P(z_i \text{ to be first pivot}) &= \frac{1}{\text{number of element}} \\ &= \frac{1}{j-i+1} \end{aligned}$$

Similarly ,

$$P(z_j \text{ to be the first pivot}) = \frac{1}{j-i+1}$$

$$\therefore P(z_i \text{ or } z_j \text{ to be the first pivot}) \\ = \frac{2}{j-i+1}$$

Theorem 7.4

The expected running time of RANDOMIZED-QUICKSORT on an input of n distinct elements is $O(n \lg n)$.

Proof The analysis uses indicator random variables (see Section 5.2). Let the n distinct elements be $z_1 < z_2 < \dots < z_n$, and for $1 \leq i < j \leq n$, define the indicator random variable $X_{ij} = I\{z_i \text{ is compared with } z_j\}$. From Lemma 7.2, each pair is compared at most once, and so we can express X as follows:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} .$$

By taking expectations of both sides and using linearity of expectation (equation (C.24) on page 1192) and Lemma 5.1 on page 130, we obtain

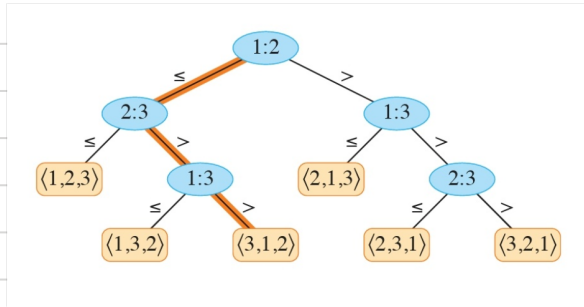
$$\begin{aligned} E[X] &= E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] && \text{(by linearity of expectation)} \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr \{z_i \text{ is compared with } z_j\} && \text{(by Lemma 5.1)} \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} && \text{(by Lemma 7.3) .} \end{aligned}$$

We can evaluate this sum using a change of variables ($k = j - i$) and the bound on the harmonic series in equation (A.9) on page 1142:

$$\begin{aligned} E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \\ &= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \\ &< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \\ &= \sum_{i=1}^{n-1} O(\lg n) \\ &= O(n \lg n) . \end{aligned}$$

Lower Bounds For Sorting:

Decision Tree Model:



For n distinct element there should be $n!$ distinct permutation as well the leaf in the tree.

Height h . For binary tree leaf $1 \leq 2^h$

$$n! \leq 2^h \quad \therefore h \geq \log(n!) = \Omega(n \lg n)$$

Beating the Lower Bound: Radix Sort

- **Linear Time Concepts:** These algorithms are not "comparison sorts" and instead use key properties (like integer values) to index into arrays
 - **Radix Sort Mechanism:**
 - Sorts on **least significant digits** first using a **stable sort** (usually Counting Sort) as a subroutine
 - **Complexity:** $\Theta(d(n + k))$ where d is digits, n is elements, and k is the range of values per digit
 - **Efficiency:** When d is a constant and $k = O(n)$, Radix Sort runs in $\Theta(n)$ **time**
 - **Stability Requirement:** Crucial point that the underlying sort must be stable for Radix Sort to work
-

